

딥러닝 파이토치 교과서 1~2장

1장 머신 러닝과 딥러닝

1.1 인공지능, 머신 러닝과 딥러닝

인공지능은 인간의 지능을 모방하여 사람이 하는 일을 컴퓨터가 할 수 있도록 하는 기술이다. 인공지능을 구현하는 방법으로 머신 러닝과 딥러닝이 있다.

- 인공지능 > 머신 러닝 > 딥러닝

머신 러닝과 딥러닝 모두 학습 모델을 제공하여 데이터를 분류할 수 있는 기술이지만, 접근 방식에 차이가 있다.

- 머신 러닝: 주어진 데이터를 인간이 먼저 처리(전처리)한다. 범용적인 목적을 위해 제작된 것으로 데이터의 특성을 스스로 추출하지 못한다.
- 딥러닝: 인간이 하던 작업을 생략한다. 대량의 데이터를 신경망에 적용하면 컴퓨터가 스스로 분석한 후 답을 찾는다.

머신 러닝과 딥러닝 차이

1. 동작 원리

- 머신 러닝: 입력 데이터에 알고리즘을 적용하여 예측을 수행한다
- 딥러닝: 정보를 전달하는 신경망을 사용하여 데이터 특징 및 관계를 해석한다

2. 재사용

- 머신 러닝: 입력 데이터를 분석하기 위해 다양한 알고리즘을 사용하며, 동일한 유형의 데이터 분석을 위한 재사용은 불가능하다
- 딥러닝: 구현된 알고리즘은 동일한 유형의 데이터를 분석하는 데 재사용된다

3. 데이터

- 머신 러닝: 일반적으로 수천 개의 데이터가 필요하다
- 딥러닝: 수백만 개 이상의 데이터가 필요하다

4. 훈련 시간

- 머신 러닝: 단시간
- 딥러닝: 장시간

5. 결과

- 머신 러닝: 일반적으로 점수 또는 분류 등 숫자 값
- 딥러닝: 출력은 점수, 텍스트, 소리 등 어떤 것이든 가능

1.2 머신 러닝이란

인공지능의 한 분야로, 컴퓨터 스스로 대용량 데이터에서 지식이나 패턴을 찾아 학습하고 예측을 수행하는 것이다. 즉, 컴퓨터가 학습할 수 있게 하는 알고리즘과 기술을 개발하는 분야다.

1.2.1 머신 러닝 학습 과정

크게 학습 단계와 예측 단계로 구분할 수 있다.

1. 학습 단계: 훈련 데이터를 머신 러닝 알고리즘에 적용하여 학습시키고, 이 학습 결과로 모형이 생성된다.
2. 예측 단계: 학습 단계에서 생성된 모형에 새로운 데이터를 적용하여 결과를 예측한다.

특성 추출이란 데이터별로 어떤 특징을 가지고 있는지 찾아내고, 그것을 토대로 데이터를 벡터로 변환하는 작업이다.

머신 러닝의 주요 구성 요소는 데이터와 모델(모형)이다.

- 데이터: 머신 러닝이 학습 모델을 만드는 데 사용하는 것으로, 훈련 데이터가 나쁘다면 실제 현상의 특성을 제대로 반영할 수 없다. 따라서 실제 데이터의 특징이 잘 반영되고 편향되지 않는 훈련 데이터를 확보하는 것이 중요하다.
- 모델: 머신 러닝의 학습 단계에서 얻은 최종 결과물로 가설이라고도 한다.

데이터는 훈련 데이터셋과 테스트 데이터셋 용도로 분리해서 사용한다. 혹은 훈련 데이터셋을 또다시 훈련 데이터셋과 검증 데이터셋으로 분리해서 사용하기도 한다. 보통 데이터의 80%는 훈련용으로, 20%는 테스트용으로 분리해서 사용한다.

모델의 학습 절차는 다음과 같다.

1. 모델(또는 가설) 선택
2. 모델 학습 및 평가
3. 평가를 바탕으로 모델 업데이트

이 세 단계를 반복하면서 주어진 문제를 가장 잘 풀 수 있는 모델을 찾는다.

훈련과 검증, 테스트 데이터셋

검증 데이터셋을 사용하는 이유는 모델 성능을 평가하기 위해서다. 훈련 데이터셋으로 모델을 학습시킨 후 모델이 잘 예측하는지 그 성능을 평가하기 위해 사용한다. 하지만 검증 용도의 데이터셋은 훈련 데이터셋의 일부를 떼어서 사용하기 때문에 학습에 사용되는 데이터셋의 양이 많지 않다면 검증 데이터셋을 사용하는 것은 좋지 않다.

모델 성능의 평가가 필요한 이유는 다음과 같다.

1. 테스트 데이터셋에 대한 성능을 가늠해 볼 수 있기 때문이다.
2. 모델 성능을 높이는 데 도움을 준다. 훈련 데이터셋에 대한 정확도는 높는데 검증 데이터셋에 대한 정확도가 낮다면 훈련 데이터셋에 과적합이 일어났을 가능성을 생각해 볼 수 있다. 이 경우 정규화를 하거나 에포크를 줄이는 방식으로 과적합을 막을 수 있다.

1.2.2 머신 러닝 학습 알고리즘

- 지도 학습: 정답이 무엇인지 컴퓨터에 알려 주고 학습시키는 방법

- 비지도 학습: 정답을 알려 주지 않고 특징이 비슷한 데이터를 클러스터링(범주화)하여 예측하는 학습 방법. 유사도 기반으로 특징이 유사한 데이터끼리 클러스터링으로 묶어서 분류한다.
- 강화 학습: 분류할 수 있는 데이터가 있는 것도 아니고 데이터가 있다고 해도 정답이 없다. 자신의 행동에 대한 보상을 받으며 학습을 진행한다. 에이전트가 변화하는 환경에 따라 다른 행동을 취하게 되며, 행동에 따라 보상을 얻는다.

자주 사용되는 알고리즘은 다음과 같다.

1. 지도 학습
 - 분류: K-최근접 이웃(KNN), 서포트 벡터 머신(SVM), 결정 트리, 로지스틱 회귀
 - 회귀: 선형 회귀
2. 비지도 학습
 - 군집: K-평균 군집화, 밀도 기반 군집 분석(DBSCAN)
 - 차원 축소: 주성분 분석(PCA)
3. 강화 학습
 - 마르코프 결정 과정(MDP)

1.3 딥러닝이란

인간의 신경망 원리를 모방한 심층 신경망 이론을 기반으로 고안된 머신 러닝 방법의 일종이다. 즉, 딥러닝이 머신 러닝과 다른 큰 차이점은 인간의 뇌를 기초로 하여 설계했다는 것이다.

인간의 뇌가 엄청난 수의 뉴런과 시냅스로 구성되어 있는 것에 착안하여 컴퓨터에 뉴런과 시냅스 개념을 적용했다.

- 수상돌기: 주변이나 다른 뉴런에서 자극을 받아들이고, 이 자극들을 전기적 신호 형태로 세포체와 축삭돌기로 보내는 역할을 한다.
- 시냅스: 신경 세포들이 이루는 연결 부위로, 한 뉴런의 축삭돌기와 다음 뉴런의 수상돌기가 만나는 부분이다.
- 축삭돌기: 다른 뉴런(수상돌기)에 신호를 전달하는 기능을 하는 뉴런의 한 부분이다. 뉴런에서 뻗어 있는 돌기 중 가장 길며, 한 개만 있다.
- 축삭말단: 전달된 전기 신호를 받아 신경 전달 물질을 시냅스 틈새로 방출한다.

1.3.1 딥러닝 학습 과정

데이터 준비부터 모델을 정의하고 사용하는 상위 레벨에서 짚고 넘어간다.

1. 데이터 준비: 초보자가 데이터를 쉽게 구할 수 있는 방법은 두 가지다. 첫째, 파이토치나 케라스에서 제공하는 데이터셋을 사용하는 것이다. 둘째, 캐글 같은 곳에 공개된 데이터를 사용하는 것이다.
2. 모델 정의: 신경망을 생성한다. 일반적으로 은닉층 개수가 많을수록 성능이 좋아지지만 과적합이 발생할 확률이 높아진다. 즉, 은닉층 개수에 따른 성능과 과적합은 서로 상충 관계에 있다.
3. 모델 컴파일: 활성화 함수, 손실 함수, 옵티마이저를 선택한다. 훈련 데이터셋 형태가 연속형이라면 평균 제곱 오차(MSE)를 사용할 수 있으며, 이진 분류라면 크로스 엔트로피를 선택한다.
4. 모델 훈련: 훈련 단계에서는 한 번에 처리할 데이터양을 지정한다. 한 번에 처리해야 할 데이터양이 많아지면 학습 속도가 느려지고 메모리 부족 문제를 야기할 수 있기 때문에 적당한 데이터양을 선택하는 것이 중요하다.

따라서 전체 훈련 데이터셋에서 일정한 묶음으로 나누어 처리할 수 있는 배치와 훈련의 횟수인 에포크 선택이 중요하다.

5. 모델 예측: 검증 데이터셋을 생성한 모델에 적용하여 실제로 예측을 진행해 보는 단계다. 이때 예측력이 낮다면 파라미터를 튜닝하거나 신경망 자체를 재설계해야 할 수도 있다.

성능에 대한 공식적인 정의는 없으며, 궁극적으로 모델 성능은 데이터가 수집된 산업 분야와 모델이 생성된 목적에 의존한다. 모델 성능이 좋다는 의미는 다음과 같은 다양한 의미로 사용할 수 있다.

- 예측을 잘합니다(정확도가 높습니다)
- 훈련 속도가 빠릅니다

딥러닝 학습 과정에서 중요한 핵심 구성 요소는 신경망과 역전파다. 딥러닝은 머신 러닝의 한 분야이기는 하지만, 심층 신경망을 사용한다는 점에서 머신 러닝과 차이가 있다. 심층 신경망에는 데이터셋의 어떤 특성들이 중요한지 스스로에게 가르쳐 줄 수 있는 기능이 있다.

또한, 가중치 값을 업데이트하기 위한 역전파가 중요하다. 특히 역전파 계산 과정에서 사용되는 미분(오차를 각 가중치로 미분)이 성능에 영향을 미치는 주요한 요소다. 하지만 파이토치 같은 프레임워크를 이용하면 역전파 알고리즘을 자동으로 처리해 주기 때문에 딥러닝 알고리즘 구현이 간단하고 편리해진다.

1.3.2 딥러닝 학습 알고리즘

활용 분야에 따라 지도 학습과 비지도 학습, 전이 학습으로 분류된다.

지도 학습으로 분류되는 알고리즘은 다음과 같다.

- 합성곱 신경망(CNN): 이미지 또는 비디오상의 객체를 식별하는 컴퓨터 비전 기술이다. 목적에 따라 이미지 분류, 이미지 인식, 이미지 분할로 분류할 수 있다.
- 순환 신경망(RNN): 시계열 데이터를 분류할 때 사용된다. 주식 데이터처럼 시간에 따른 데이터가 있을 때 사용하지만, 역전파 과정에서 기울기 소멸 문제가 발생하는 단점이 있다.
- LSTM: 기울기 소멸 문제를 개선하고자 게이트를 세 개 추가한 것이다. 망각 게이트(과거 정보를 잊기 위한 게이트), 입력 게이트(현재 정보를 기억하기 위한 게이트)와 출력 게이트(최종 결과를 위한 게이트)를 도입하여 기울기 소멸 문제를 해결했다.

비지도 학습으로 분류되는 알고리즘은 다음과 같다.

- 워드 임베딩: 자연어를 컴퓨터가 이해하고 효율적으로 처리하게 하려면 컴퓨터가 이해할 수 있도록 자연어를 적절히 변환하는 것이 필요하다. 이때 워드 임베딩 기술을 이용하여 단어를 벡터로 표현한다. 단어 의미를 벡터화하는 워드투벡터(Word2Vec)와 글로브(GloVe)를 가장 많이 사용하고 있다.
- 군집: 아무런 정보가 없는 상태에서 데이터를 분류하는 방법이다. 한 클러스터 안의 데이터는 매우 비슷하게 구성하고 다른 클러스터의 데이터와 구분되도록 나누는 것이 목표다. 머신 러닝의 군집과 다르지 않지만, 딥러닝과 함께 사용하면 모델 성능을 높일 수 있다.

2장 실습 환경 설정과 파이토치 기초

2.1 파이토치 개요

파이토치는 2017년 초에 공개된 딥러닝 프레임워크로 루아 언어로 개발되었던 토치를 페이스북에서 파이썬 버전으로 내놓은 것이다. 토치는 파이썬의 넘파이 라이브러리처럼 과학 연산을 위한 라이브러리로 공개되었지만 이후 발전을 거듭하면서 딥러닝 프레임워크로 발전했다.

파이토치 공식 튜토리얼에서는 파이썬 기반의 과학 연산 패키지로 다음 두 집단을 대상으로 한다고 언급한다.

- 넘파이를 대체하면서 GPU를 이용한 연산이 필요한 경우
- 최대한의 유연성과 속도를 제공하는 딥러닝 연구 플랫폼이 필요한 경우

무엇보다 주목받는 이유 중 하나는 간결하고 빠른 구현성에 있다.

2.1.1 파이토치 특징 및 장점

파이토치의 특징은 다음과 같이 한마디로 특징 지을 수 있다.

- GPU에서 텐서 조작 및 동적 신경망 구축이 가능한 프레임워크

GPU, 텐서, 동적 신경망의 의미는 다음과 같다.

1. GPU: 연산 속도를 빠르게 하는 역할을 한다.

- 딥러닝에서는 기울기를 계산할 때 미분을 쓰는데, GPU를 사용하면 빠른 계산이 가능하다.
- 내부적으로 CUDA, cuDNN이라는 API를 통해 GPU를 연산에 사용할 수 있다.
- 병렬 연산에서 GPU의 속도는 CPU의 속도보다 훨씬 빠르므로 딥러닝 학습에서 GPU 사용은 필수라고 할 수 있다.

2. 텐서: 파이토치에서 텐서 의미는 다음과 같다.

- 텐서는 파이토치의 데이터 형태다.
- 텐서는 단일 데이터 형식으로 된 자료들의 다차원 행렬이다.
- 텐서는 간단한 명령어(변수 뒤에 `.cuda()`를 추가)를 사용해서 GPU로 연산을 수행하게 할 수 있다.

3. 동적 신경망: 훈련을 반복할 때마다 네트워크 변경이 가능한 신경망을 의미한다.

- 예를 들어 학습 중에 은닉층을 추가하거나 제거하는 등 모델의 네트워크 조작이 가능하다.
- 연산 그래프를 정의하는 것과 동시에 값도 초기화되는 'Define by Run' 방식을 사용한다. 따라서 연산 그래프와 연산을 분리해서 생각할 필요가 없기 때문에 코드를 이해하기 쉽다.

연산 그래프는 방향성이 있으며 변수(텐서)를 의미하는 노드와 연산(곱하기, 더하기)을 담당하는 엣지로 구성된다. 신경망은 연산 그래프를 이용하여 계산을 수행한다. 즉, 네트워크가 학습될 때 손실 함수의 기울기가 가중치와 바이어스를 기반으로 계산되며, 이후 경사 하강법을 사용하여 가중치가 업데이트된다.

파이토치는 효율적인 계산, 낮은 CPU 활용, 직관적인 인터페이스와 낮은 진입 장벽 등을 장점으로 꼽을 수 있다.

1. 단순함(효율적인 계산)

- 파이썬 환경과 쉽게 통합할 수 있다.

- 디버깅이 직관적이고 간결하다.
2. 성능(낮은 CPU 활용)
- 모델 훈련을 위한 CPU 사용률이 텐서플로와 비교하여 낮다.
 - 학습 및 추론 속도가 빠르고 다루기 쉽다.
3. 직관적인 인터페이스
- 텐서플로처럼 잦은 API 변경이 없어 배우기 쉽다.

벡터, 행렬, 텐서

인공지능에서 데이터는 벡터로 표현된다. 벡터는 [1.0, 1.1, 1.2]처럼 숫자들의 리스트로, 1차원 배열 형태다. 행렬은 행과 열로 표현되는 2차원 배열 형태다. 이때 가로줄을 행이라고 하며, 세로줄을 열이라고 한다. 마지막으로 텐서는 3차원 이상의 배열 형태다.

- 1차원 축(행) = axis 0 = 벡터
- 2차원 축(열) = axis 1 = 행렬
- 3차원 축(채널) = axis 2 = 텐서

2.1.2 파이토치의 아키텍처

크게 세 개의 계층으로 나누어 설명할 수 있다. 가장 상위 계층은 파이토치 API가 위치해 있으며 그 아래에는 파이토치 엔진이 있다. 그리고 마지막으로 가장 아래에는 텐서에 대한 연산을 처리한다.

파이토치 API 계층에서는 사용자가 이해하기 쉬운 API를 제공하여 텐서에 대한 처리와 신경망을 구축하고 훈련할 수 있도록 돕는다. 이 계층에서는 사용자 인터페이스를 제공하지만 실제 계산은 수행하지 않는다. 그 대신 C++로 작성된 파이토치 엔진으로 그 작업을 전달하는 역할만 한다.

- torch: GPU를 지원하는 텐서 패키지. 다차원 텐서를 기반으로 다양한 수학적 연산이 가능하도록 한다. 특히 CPU뿐만 아니라 GPU에서 연산이 가능하므로 빠른 속도로 많은 양의 계산을 할 수 있다.
- torch.autograd: 자동 미분 패키지. 일반적으로 신경망에 사소한 변경이 있다면 신경망 구축을 처음부터 다시 시작해야 하지만, 파이토치는 '자동 미분'이라는 기술을 채택하여 미분 계산을 효율적으로 처리한다.
- torch.nn: 신경망 구축 및 훈련 패키지. 합성곱 신경망, 순환 신경망, 정규화 등이 포함되어 손쉽게 신경망을 구축하고 학습시킬 수 있다.
- torch.multiprocessing: 파이썬 멀티프로세싱 패키지. 파이토치에서 사용하는 프로세스 전반에 걸쳐 텐서의 메모리 공유가 가능하다. 따라서 서로 다른 프로세스에서 동일한 데이터(텐서)에 대한 접근 및 사용이 가능하다.
- torch.utils: DataLoader 및 기타 유틸리티를 제공하는 패키지. 모델에 데이터를 제공하기 위한 torch.utils.data.DataLoader 모듈을 주로 사용한다. 또한, 병목 현상을 디버깅하기 위한 torch.utils.bottleneck, 모델 또는 모델의 일부를 검사하기 위한 torch.utils.checkpoint 등의 모듈도 있다.

파이토치 엔진은 Autograd C++, Aten C++, JIT C++, Python API로 구성되어 있다. Autograd C++는 가중치, 바이어스를 업데이트하는 과정에서 필요한 미분을 자동으로 계산해 주는 역할을 하며, Aten C++는 C++ 텐서 라이브러리를 제공하며, JIT C++는 계산을 최적화하기 위한 JIT 컴파일러다.

가장 아래 계층에 속하는 C 또는 CUDA 패키지는 상위의 API에서 할당된 거의 모든 계산을 수행한다. 여기에서 제공되는 패키지는 CPU와 GPU(TH(토치), THC(토치 CUDA))를 이용하여 효율적인 데이터 구조, 다차원 텐서에 대한 연산을 처리한다.

텐서를 메모리에 저장하기

텐서는 그것이 1차원이든 N차원이든 메모리에 저장할 때는 1차원 배열 형태가 된다. 즉, 1차원 배열 형태여야만 메모리에 저장할 수 있다. 그리고 변환된 1차원 배열을 스토리지라고 한다. 스토리지를 이해하기 위해서는 오프셋과 스트라이드 개념을 알아야 한다.

- 오프셋: 텐서에서 첫 번째 요소가 스토리지에 저장된 인덱스다.
- 스트라이드: 각 차원에 따라 다음 요소를 얻기 위해 건너뛰기가 필요한 스토리지의 요소 개수다. 즉, 스트라이드는 메모리에서의 텐서 레이아웃을 표현하는 것으로 이해하면 된다.

오프셋과 스트라이드는 데이터 자체가 아닌 행렬/텐서를 구분하기 위해 사용한다.

2.2 파이토치 기초 문법

파이토치는 텐서로 시작해서 텐서로 끝난다고 해도 과언이 아니다. 따라서 텐서를 잘 다룰 수 있어야 문제 해결이 가능하고 신경망에서 데이터 입력과 출력도 제어할 수 있다.

2.2.1 텐서 다루기

텐서 생성 및 변환

- 텐서는 파이토치의 가장 기본이 되는 데이터 구조다. 넘파이의 ndarray와 비슷하며 GPU에서의 연산도 가능하다.

텐서의 인덱스 조작

- 텐서는 넘파이의 ndarray를 조작하는 것과 유사하게 동작하기 때문에 배열처럼 인덱스를 바로 지정하거나 슬라이스 등을 사용할 수 있다.
- 텐서의 자료형은 다음과 같다.
- torch.FloatTensor: 32비트의 부동 소수점
- torch.DoubleTensor: 64비트의 부동 소수점
- torch.LongTensor: 64비트의 부호가 있는 정수

텐서 연산 및 차원 조작

- 텐서는 넘파이의 ndarray처럼 다양한 수학 연산이 가능하며, GPU를 사용하면 더 빠르게 연산할 수 있다. 참고로 텐서 간의 타입이 다르면 연산이 불가능하다.
- 텐서의 차원을 변경하는 가장 대표적인 방법은 view를 이용하는 것이다. 이외에도 텐서를 결합하는 stack, cat과 차원을 교환하는 t, transpose도 사용된다.
- view는 넘파이의 reshape과 유사하며 cat은 다른 길이의 텐서를 하나로 병합할 때 사용한다. 또한, transpose는 행렬의 전치 외에도 차원의 순서를 변경할 때도 사용된다.

2.2.2 데이터 준비

데이터 호출에는 파이썬 라이브러리(판다스)를 이용하는 방법과 파이토치에서 제공하는 데이터를 이용하는 방법이 있다. 데이터가 이미지일 경우 분산된 파일에서 데이터를 읽은 후 전처리를 하고 배치 단위로 분할하여 처리한다. 데이터가 텍스트일 경우 임베딩 과정을 거쳐 서로 다른 길이의 시퀀스를 배치 단위로 분할하여 처리한다.

1. 단순하게 파일을 불러와서 사용

- 판다스 라이브러리를 이용하여 JSON, PDF, CSV 등의 파일을 불러오는 방법이다. 데이터가 복잡하지 않은 형태라면 단순하고 유용하게 사용될 수 있다.

2. 커스텀 데이터셋을 만들어서 사용

- 딥러닝은 기본적으로 대량의 데이터를 이용하여 모델을 학습시킨다. 하지만 데이터를 한 번에 메모리에 불러와서 훈련시키면 시간과 비용 측면에서 효율적이지 않다. 따라서 데이터를 한 번에 다 부르지 않고 조금씩 나누어 불러서 사용하는 방식이 커스텀 데이터셋이다.
- CustomDataset 클래스를 구현하기 위해서는 다음 형태를 취해야 한다.
- `__init__(self)`: 필요한 변수를 선언하고, 데이터셋의 전처리를 해 주는 함수
- `__len__(self)`: 데이터셋의 길이. 즉, 총 샘플의 수를 가져오는 함수
- `__getitem__(self, index)`: 데이터셋에서 특정 데이터를 가져오는 함수. 이때 반환되는 값은 텐서의 형태를 취해야 한다

3. 파이토치에서 제공하는 데이터셋 사용

- 토치비전은 파이토치에서 제공하는 데이터셋들이 모여 있는 패키지다. MNIST, ImageNet을 포함한 유명한 데이터셋들을 제공하고 있다.

데이터로더 객체는 학습에 사용될 데이터 전체를 보관했다가 모델 학습을 할 때 배치 크기만큼 데이터를 꺼내서 사용한다. 이때 주의할 것은 데이터를 미리 잘라 놓는 것이 아니라 내부적으로 반복자에 포함된 인덱스를 이용하여 배치 크기만큼 데이터를 반환한다는 것이다.

2.2.3 모델 정의

파이토치에서 모델을 정의하기 위해서는 모듈을 상속한 클래스를 사용한다.

- 계층: 모듈 또는 모듈을 구성하는 한 개의 계층으로 합성곱층, 선형 계층 등이 있다.
- 모듈: 한 개 이상의 계층이 모여서 구성된 것으로, 모듈이 모여 새로운 모듈을 만들 수도 있다.
- 모델: 최종적으로 원하는 네트워크로, 한 개의 모듈이 모델이 될 수도 있다.

모델을 정의하는 방법은 다음과 같다.

1. 단순 신경망을 정의하는 방법

- nn.Module을 상속받지 않는 매우 단순한 모델을 만들 때 사용한다. 구현이 쉽고 단순하다는 장점이 있다.

2. nn.Module()을 상속하여 정의하는 방법

- nn.Module을 상속받는 모델은 기본적으로 `__init__()` 과 `forward()` 함수를 포함한다. `__init__()` 에서는 모델에서 사용될 모듈, 활성화 함수 등을 정의하고, `forward()` 함수에서는 모델에서 실행되어야 하는 연산을 정의한다.

3. Sequential 신경망을 정의하는 방법

- nn.Sequential을 사용하면 `__init__()` 에서 사용할 네트워크 모델들을 정의해 줄 뿐만 아니라 `forward()` 함수에서는 모델에서 실행되어야 할 계산을 좀 더 가독성이 뛰어나게 코드로 작성할 수 있다. 또한, Sequential 객체는 그 안에 포함된 각 모듈을 순차적으로 실행해 준다. nn.Sequential은 모델의 계층이 복잡할수록 효과가 뛰어나다.

4. 함수로 신경망을 정의하는 방법

- Sequential을 이용하는 것과 동일하지만, 함수로 선언할 경우 변수에 저장해 놓은 계층들을 재사용할 수 있는 장점이 있다. 하지만 모델이 복잡해지는 단점도 있다. 참고로 복잡한 모델의 경우에는 함수를 이용하는 것보다는 nn.Module()을 상속받아 사용하는 것이 편리하다.

model.modules()는 모델의 네트워크에 대한 모든 노드를 반환하며, model.children()은 같은 수준의 하위 노드를 반환한다.

2.2.4 모델의 파라미터 정의

모델을 학습하기 전에 필요한 파라미터들을 정의한다.

- 손실 함수: 학습하는 동안 출력과 실제 값 사이의 오차를 측정한다. 즉, $w x + b$ 를 계산한 값과 실제 값인 y 의 오차를 구해서 모델의 정확성을 측정한다. 손실 함수로 많이 사용되는 것은 다음과 같다.
- BCELoss: 이진 분류를 위해 사용
- CrossEntropyLoss: 다중 클래스 분류를 위해 사용
- MSELoss: 회귀 모델에서 사용
- 옵티마이저: 데이터와 손실 함수를 바탕으로 모델의 업데이트 방법을 결정한다. 다음은 옵티마이저의 주요 특성이다.
- optimizer는 step() 메서드를 통해 전달받은 파라미터를 업데이트한다.
- 모델의 파라미터별로 다른 기준(예: 학습률)을 적용시킬 수 있다.
- torch.optim.Optimizer(params, defaults)는 모든 옵티마이저의 기본이 되는 클래스다.
- zero_grad() 메서드는 옵티마이저에 사용된 파라미터들의 기울기를 0으로 만든다.
- torch.optim.lr_scheduler는 에포크에 따라 학습률을 조절할 수 있다.
- 학습률 스케줄러: 미리 지정한 횟수의 에포크를 지날 때마다 학습률을 감소시켜 준다. 학습률 스케줄러를 이용하면 학습 초기에는 빠른 학습을 진행하다가 전역 최소점 근처에 다다르면 학습률을 줄여서 최적점을 찾아갈 수 있도록 해 준다.
- 지표: 훈련과 테스트 단계를 모니터링한다.

전역 최소점과 최적점

손실 함수는 실제 값과 예측 값 차이를 수치화해 주는 함수다. 이 오차 값이 클수록 손실 함수의 값이 크고, 오차 값이 작을수록 손실 함수의 값이 작아진다. 그리고 이 손실 함수의 값을 최소화하는 가중치와 바이어스를 찾는 것이 학습 목표다.

전역 최소점은 오차가 가장 작을 때의 값을 의미하므로 우리가 최종적으로 찾고자 하는 것, 즉 최적점이라고 할 수 있다. 그리고 지역 최소점은 전역 최소점을 찾아가는 과정에서 만나는 골짜기 같은 것으로 옵티마이저가 지역 최소점에서 학습을 멈추면 최소값을 갖는 오차를 찾을 수 없는 문제가 발생한다.

2.2.5 모델 훈련

앞서 만들어 둔 데이터로 모델을 학습시킨다. 이때 학습을 시킨다는 것은 $y = wx + b$ 라는 함수에서 w 와 b 의 적절한 값을 찾는다는 의미다. w 와 b 에 임의의 값을 적용하여 시작하며 오차가 줄어들어 전역 최소점에 이를 때까지 파라미터(w, b)를 계속 수정한다.

가장 먼저 필요한 절차가 `optimizer.zero_grad()` 메서드를 이용하여 기울기를 초기화하는 것이다. 파이토치는 기울기 값을 계산하기 위해 `loss.backward()` 메서드를 이용하는데, 이것을 사용하면 새로운 기울기 값이 이전 기울기 값에 누적하여 계산된다. 이 방법은 순환 신경망 모델을 구현할 때 효과적이지만 누적 계산이 필요하지 않는 모델에 대해서는 불필요하다. 따라서 기울기 값에 대해 누적 계산이 필요하지 않을 때는 입력 값을 모델에 적용하기 전에 `optimizer.zero_grad()` 메서드를 호출하여 미분 값이 누적되지 않게 초기화해 주어야 한다.

파이토치 학습 절차는 다음과 같다.

1. 모델, 손실 함수, 옵티마이저 정의
2. `optimizer.zero_grad()`: 전방향 학습, 기울기 초기화
3. `output = model(input)`: 출력 계산
4. `loss = loss_fn(output, target)`: 오차 계산
5. `loss.backward()`: 역전파 학습
6. `optimizer.step()`: 기울기 업데이트

`loss.backward()`는 배치가 반복될 때마다 오차가 중첩적으로 쌓이게 되므로 매번 `zero_grad()`를 사용하여 미분 값을 0으로 초기화한다.

2.2.6 모델 평가

주어진 테스트 데이터셋을 사용하여 모델을 평가한다. 모델에 대한 평가는 함수와 모듈을 이용하는 두 가지 방법이 있으며, `torchmetrics` 패키지를 이용한다.

혹은 사이킷런에서 제공하는 혼동 행렬을 이용하는 방법도 고려해 볼 수 있다. 사이킷런의 `metrics` 모듈에서 제공하는 `confusion_matrix`, `accuracy_score`와 `classification_report` 클래스를 이용하면 쉽게 정확도를 찾을 수 있다.

2.2.7 훈련 과정 모니터링

파이토치로 머신 러닝/딥러닝 모델을 만들어 학습해 보면 학습이 진행되는 과정에서 각 파라미터에 어떤 값들이 어떻게 변화하는지 모니터링하기 어렵다. 이때 텐서보드를 이용하면 학습에 사용되는 각종 파라미터 값이 어떻게 변화하는지 손쉽게 시각화하여 살펴볼 수 있으며 성능을 추적하거나 평가하는 용도로도 사용할 수 있다.

파이토치에서 텐서보드를 사용하는 방법은 다음과 같다.

1. 텐서보드를 설정한다.
2. 텐서보드에 기록한다.
3. 텐서보드를 사용하여 모델 구조를 살펴본다.

`model.train()`과 `model.eval()`

- `model.train()`: 훈련 데이터셋에 사용하며 모델 훈련이 진행될 것임을 알린다. 이때 드롭아웃이 활성화된다.

- `model.eval()`: 모델을 평가할 때는 모든 노드를 사용하겠다는 의미로 검증과 테스트 데이터셋에 사용한다.

`model.train()`과 `model.eval()`을 선언해야 모델의 정확도를 높일 수 있다.

`model.eval()`에서 `with torch.no_grad()`를 사용하는 이유는 다음과 같다. 파이토치는 모든 연산과 기울기 값을 저장한다. 하지만 검증(혹은 테스트) 과정에서는 역전파가 필요하지 않기 때문에 `with torch.no_grad()`를 사용하여 기울기 값을 저장하지 않도록 한다. 이와 같은 과정을 통해 기울기 값을 저장하고 기록하는 데 필요한 메모리와 연산 시간을 줄일 수 있다.

2.3 데이터 전처리

딥러닝은 통계 알고리즘을 기반으로 하기 때문에 단어를 숫자(텐서)로 변환해야 한다. 가장 먼저 필요한 전처리는 데이터를 파악하는 것이다. 주어진 데이터의 형태를 파악한 후 숫자로 변환해 주어야 한다.

범주형 데이터를 텐서로 변환하기 위해 다음과 같은 절차가 필요하다.

- 범주형 데이터 → 넘파이 배열 → 텐서

즉, 파이토치로 모델을 학습시키기 위해서는 텐서 형태로 변환해야 하는데, 넘파이 배열을 통해 텐서를 생성할 수 있다.

워드 임베딩은 유사한 단어끼리 유사하게 인코딩되도록 표현하는 방법이다. 또한, 높은 차원의 임베딩일수록 단어 간의 세부적인 관계를 잘 파악할 수 있다. 따라서 단일 숫자로 변환된 넘파이 배열을 N차원으로 변경하여 사용한다. 임베딩 크기에 대한 정확한 규칙은 없지만, 칼럼의 고유 값 수를 2로 나누는 것을 많이 사용한다.

2.4 모델 성능 평가 지표

정확도를 확인하기 전에 필요한 용어들은 다음과 같다.

- True Positive: 모델(분류기)이 '1'이라고 예측했는데 실제 값도 '1'인 경우다.
- True Negative: 모델(분류기)이 '0'이라고 예측했는데 실제 값도 '0'인 경우다.
- False Positive: 모델(분류기)이 '1'이라고 예측했는데 실제 값은 '0'인 경우로, Type I 오류라고도 한다.
- False Negative: 모델(분류기)이 '0'이라고 예측했는데 실제 값은 '1'인 경우로, Type II 오류라고도 한다.

이러한 용어들을 사용하여 정확도, 재현율, 정밀도, F1-스코어에 대해 정리하면 다음과 같다.

1. 정확도: 전체 예측 건수에서 정답을 맞힌 건수의 비율이다. 이때 맞힌 정답이 긍정이든 부정이든 상관없다.
 - $(\text{True Positive} + \text{True Negative}) / (\text{True Positive} + \text{True Negative} + \text{False Positive} + \text{False Negative})$
2. 재현율: 실제로 정답이 1이라고 할 때 모델(분류기)도 1로 예측한 비율이다. 따라서 처음부터 데이터가 1일 확률이 적을 때 사용하면 좋다.
 - $\text{True Positive} / (\text{True Positive} + \text{False Negative})$
3. 정밀도: 모델(분류기)이 1이라고 예측한 것 중에서 실제로 정답이 1인 비율이다.
 - $\text{True Positive} / (\text{True Positive} + \text{False Positive})$

4. F1-스코어: 일반적으로 정밀도와 재현율은 트레이드오프 관계다. 정밀도가 높으면 재현율이 낮고, 재현율이 높으면 정밀도가 낮아진다. 이러한 트레이드오프 문제를 해결하려고 정밀도와 재현율의 조화 평균을 이용한 것이 F1-스코어 평가다.

- $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$